AI Enemy Bot Game — Intelligent Enemy Characters that React to the Player

Mini Project: AI-based Topics (Search Algorithms)

Aim

To design and implement an intelligent enemy character in a 2D game environment that dynamically reacts to the player's position and actions, using a Finite State Machine (FSM) for decision-making combined with the A* search algorithm for pathfinding.

Algorithm

1. **Initialize** the grid, wall obstacles, player position, and enemy position with **state = PATROL**.
2. **Repeat** each game tick:
   a. Read the player's current position.
   b. **Decide state** (FSM transition, checked in priority order):
      • If enemy.health < 30 → **FLEE**
      • Else if distance(enemy, player) ≤ 1 → **ATTACK**
      • Else if distance(enemy, player) ≤ 6 → **CHASE**
      • Else → **PATROL**
   c. **Act** on the current state:
      • **CHASE**: run A* from enemy position to player position; move one step along the path
      • **FLEE**: pick the farthest safe cell from the player; run A* to it; move one step
      • **PATROL**: run A* toward the next fixed waypoint; move one step
      • **ATTACK**: stay in place; deal damage to the player each tick
   d. If the player is adjacent and attacks, reduce enemy health.
   e. Check win/loss condition (either health ≤ 0).
3. **Until** game over.

A* Subroutine

Standard f(n) = g(n) + h(n) search with Manhattan-distance heuristic h. Expands the lowest-f node from a priority queue until the goal is popped, then reconstructs the path via parent pointers.

Program

The full pygame implementation below contains the FSM (EnemyBot class) and the A* search function. Run locally with: `pip install pygame` then `python3 ai_enemy_bot_game.py`.

```
"""
AI Enemy Bot Game — Intelligent Enemy Characters that React to the Player
--------------------------------------------------------------------------
Mini Project: AI-based Topics

AI Design:
  - Finite State Machine (FSM): PATROL -> CHASE -> ATTACK -> FLEE
  - A* Search: used by the enemy to navigate the grid toward the player
    (CHASE) or away from the player (FLEE), avoiding walls.

Controls:
  Arrow keys : move the player
  Space      : attack the enemy if adjacent
  R          : restart after game over
  Esc        : quit

Requirements:
  pip install pygame
"""

import heapq
import random
import pygame

# ---------------------------------------------------------------------------
# Configuration
# ---------------------------------------------------------------------------

CELL_SIZE = 40
COLS, ROWS = 16, 12
WIDTH, HEIGHT = COLS * CELL_SIZE, ROWS * CELL_SIZE + 60  # extra space for HUD
FPS = 8  # grid-based movement -> low FPS = one move per tick, easy to follow

DETECTION_RADIUS = 6
ATTACK_RANGE = 1
FLEE_HEALTH_THRESHOLD = 30
PLAYER_DAMAGE = 15
ENEMY_DAMAGE = 10

WHITE, BLACK, GRAY = (245, 245, 245), (20, 20, 20), (90, 90, 90)
GREEN, RED, YELLOW, BLUE = (76, 175, 80), (244, 67, 54), (255, 213, 79), (66, 135, 245)

STATE_COLORS = {
    "PATROL": (120, 170, 255),
    "CHASE": (255, 152, 0),
    "ATTACK": (244, 67, 54),
    "FLEE": (156, 39, 176),
}


# ---------------------------------------------------------------------------
# Grid / walls
# ---------------------------------------------------------------------------

def build_walls():
    """A handful of fixed obstacles so pathfinding has something to solve."""
    walls = set()
    for r in range(3, 9):
        walls.add((r, 6))
    for c in range(8, 13):
        walls.add((3, c))
    for r in range(5, 11):
        walls.add((r, 11))
    return walls


def in_bounds(cell):
    r, c = cell
    return 0 <= r < ROWS and 0 <= c < COLS


def neighbors(cell, walls):
    r, c = cell
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
```

```
        nxt = (r + dr, c + dc)
        if in_bounds(nxt) and nxt not in walls:
            yield nxt


def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])


# ---------------------------------------------------------------------------
# A* Search — used by the enemy for both CHASE and FLEE navigation
# ---------------------------------------------------------------------------

def a_star(start, goal, walls):
    if start == goal:
        return []

    frontier = [(manhattan(start, goal), 0, start)]
    came_from = {}
    g_score = {start: 0}

    while frontier:
        _, g, current = heapq.heappop(frontier)

        if current == goal:
            path = [current]
            while path[-1] != start:
                path.append(came_from[path[-1]])
            path.reverse()
            return path[1:]  # exclude the starting cell

        for nxt in neighbors(current, walls):
            tentative_g = g + 1
            if nxt not in g_score or tentative_g < g_score[nxt]:
                g_score[nxt] = tentative_g
                came_from[nxt] = current
                heapq.heappush(frontier, (tentative_g + manhattan(nxt, goal), tentative_g, nxt))

    return []  # no path found


def farthest_cell_from(target, walls, sample_size=40):
    """Used for FLEE: sample open cells and pick the one farthest from target."""
    candidates = [(r, c) for r in range(ROWS) for c in range(COLS)
                  if (r, c) not in walls]
    candidates = random.sample(candidates, min(sample_size, len(candidates)))
    return max(candidates, key=lambda cell: manhattan(cell, target))


# ---------------------------------------------------------------------------
# Enemy — Finite State Machine
# ---------------------------------------------------------------------------

class EnemyBot:
    def __init__(self, pos, patrol_route, walls):
        self.pos = pos
        self.walls = walls
        self.health = 100
        self.state = "PATROL"
        self.path = []
        self.patrol_route = patrol_route
        self.patrol_index = 0

    def decide_state(self, player_pos):
        """FSM transition logic — reacts to the player's position and own health."""
        dist_to_player = manhattan(self.pos, player_pos)

        if self.health < FLEE_HEALTH_THRESHOLD:
            new_state = "FLEE"
        elif dist_to_player <= ATTACK_RANGE:
            new_state = "ATTACK"
        elif dist_to_player <= DETECTION_RADIUS:
            new_state = "CHASE"
        else:
            new_state = "PATROL"

        if new_state != self.state:
            self.state = new_state
            self.path = []  # force a path recompute on state change
        return self.state

    def act(self, player_pos):
```

```
        """Executes one tick of behavior for the current state."""
        if self.state == "ATTACK":
            return ENEMY_DAMAGE  # deals damage, does not move

        if self.state == "CHASE":
            if not self.path:
                self.path = a_star(self.pos, player_pos, self.walls)
            elif random.random() < 0.3:  # occasionally refresh path as player moves
                self.path = a_star(self.pos, player_pos, self.walls)
            if self.path:
                self.pos = self.path.pop(0)

        elif self.state == "FLEE":
            if not self.path:
                target = farthest_cell_from(player_pos, self.walls)
                self.path = a_star(self.pos, target, self.walls)
            if self.path:
                self.pos = self.path.pop(0)

        else:  # PATROL
            if not self.path:
                target = self.patrol_route[self.patrol_index]
                if self.pos == target:
                    self.patrol_index = (self.patrol_index + 1) % len(self.patrol_route)
                    target = self.patrol_route[self.patrol_index]
                self.path = a_star(self.pos, target, self.walls)
            if self.path:
                self.pos = self.path.pop(0)

        return 0


# ---------------------------------------------------------------------------
# Game
# ---------------------------------------------------------------------------

def draw_grid(screen, walls):
    for r in range(ROWS):
        for c in range(COLS):
            rect = pygame.Rect(c * CELL_SIZE, r * CELL_SIZE, CELL_SIZE, CELL_SIZE)
            pygame.draw.rect(screen, WHITE if (r, c) not in walls else GRAY, rect)
            pygame.draw.rect(screen, (220, 220, 220), rect, 1)


def draw_cell(screen, pos, color, margin=6):
    r, c = pos
    rect = pygame.Rect(c * CELL_SIZE + margin, r * CELL_SIZE + margin,
                       CELL_SIZE - 2 * margin, CELL_SIZE - 2 * margin)
    pygame.draw.rect(screen, color, rect, border_radius=6)


def draw_hud(screen, font, player_health, enemy):
    hud_y = ROWS * CELL_SIZE
    pygame.draw.rect(screen, BLACK, (0, hud_y, WIDTH, 60))
    text = font.render(
        f"Player HP: {max(player_health, 0):<4}   Enemy HP: {max(enemy.health, 0):<4}   Enemy state: {enemy.state}",
        True, WHITE)
    screen.blit(text, (10, hud_y + 18))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("AI Enemy Bot Game")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 18)

    walls = build_walls()
    patrol_route = [(1, 1), (1, 14), (10, 14), (10, 1)]

    def new_game():
        return {"player_pos": (10, 1), "player_health": 100,
                "enemy": EnemyBot((1, 14), patrol_route, walls), "game_over": False, "message": ""}

    state = new_game()
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
```

```
                if event.key == pygame.K_ESCAPE:
                    running = False
                elif event.key == pygame.K_r and state["game_over"]:
                    state = new_game()
                elif not state["game_over"]:
                    r, c = state["player_pos"]
                    new_pos = state["player_pos"]
                    if event.key == pygame.K_UP:
                        new_pos = (r - 1, c)
                    elif event.key == pygame.K_DOWN:
                        new_pos = (r + 1, c)
                    elif event.key == pygame.K_LEFT:
                        new_pos = (r, c - 1)
                    elif event.key == pygame.K_RIGHT:
                        new_pos = (r, c + 1)
                    elif event.key == pygame.K_SPACE:
                        if manhattan(state["player_pos"], state["enemy"].pos) <= ATTACK_RANGE:
                            state["enemy"].health -= PLAYER_DAMAGE

                    if in_bounds(new_pos) and new_pos not in walls:
                        state["player_pos"] = new_pos

                    # --- Enemy AI reacts to the player's new position ---
                    enemy = state["enemy"]
                    enemy.decide_state(state["player_pos"])
                    dmg = enemy.act(state["player_pos"])
                    state["player_health"] -= dmg

                    if state["player_health"] <= 0:
                        state["game_over"] = True
                        state["message"] = "You were defeated! Press R to restart."
                    elif enemy.health <= 0:
                        state["game_over"] = True
                        state["message"] = "Enemy defeated! Press R to restart."

        screen.fill(BLACK)
        draw_grid(screen, walls)
        for w in walls:
            draw_cell(screen, w, GRAY, margin=0)
        draw_cell(screen, state["player_pos"], BLUE)
        draw_cell(screen, state["enemy"].pos, STATE_COLORS[state["enemy"].state])
        draw_hud(screen, font, state["player_health"], state["enemy"])

        if state["game_over"]:
            msg = font.render(state["message"], True, YELLOW)
            screen.blit(msg, (10, ROWS * CELL_SIZE - 30))

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

Output

```
Turn  PlayerPos   EnemyPos    State     Changed  PlayerHP  EnemyHP
-----------------------------------------------------------------------
1     (9, 1)      (1, 13)     PATROL    False    100       100
2     (8, 1)      (1, 12)     PATROL    False    100       100
3     (7, 1)      (1, 11)     PATROL    False    100       100
4     (6, 1)      (1, 10)     PATROL    False    100       100
5     (5, 1)      (1, 9)      PATROL    False    100       100
6     (4, 1)      (1, 8)      PATROL    False    100       100
7     (3, 1)      (1, 7)      PATROL    False    100       100
8     (2, 1)      (1, 6)      PATROL    False    100       100
9     (1, 1)      (1, 5)      CHASE     True     100       100
10    (1, 2)      (1, 4)      CHASE     False    100       100
11    (1, 3)      (1, 4)      ATTACK    True     90        85
12    (1, 4)      (1, 4)      ATTACK    False    80        70
13    (1, 4)      (1, 4)      ATTACK    False    70        55
14    (1, 4)      (1, 4)      ATTACK    False    60        40
15    (1, 4)      (1, 4)      ATTACK    False    50        25
16    (1, 4)      (1, 5)      FLEE      True     50        25
17    (1, 4)      (1, 6)      FLEE      False    50        25
18    (1, 4)      (1, 7)      FLEE      False    50        25
19    (1, 4)      (1, 8)      FLEE      False    50        25
20    (1, 4)      (1, 9)      FLEE      False    50        25
21    (1, 4)      (1, 10)     FLEE      False    50        25

Total turns simulated: 21
Total nodes expanded by A* across all recomputations: 124
Final player HP: 50, Final enemy HP: 25
Distinct states observed: ['ATTACK', 'CHASE', 'FLEE', 'PATROL']
```